StaffSync Human Resource Management System

Full Stack Development MERN Stack Capstone Project

Prepared by Sanjana Tiwari
Date 25 October 2026
Technology MongoDB, Express, React and Node.js

Abstract

StaffSync is a working web application for employee records, leave requests, attendance, company announcements and team training. It provides distinct HR, manager and employee experiences backed by a shared MongoDB database. Role checks and team scoping run on the server, so hiding an interface control is not the only access boundary.

The application includes JWT sign-in, salted password hashing, profile administration, digital leave review, daily attendance, announcement editing, training assignment, private notifications, CSV exports and responsive dashboards. The local demo uses a real MongoDB process with WiredTiger data files retained on disk. A standard connection URI supports an independently managed MongoDB Community or Atlas deployment.

Implementation outcome

The application was built and tested on localhost. Fourteen automated backend tests passed. Browser workflows verified the HR-to-employee-to-manager leave cycle, attendance, directory editing, training, notifications, CSV export and pagination. Screenshots in this report were captured from the implemented application. Demonstration identities and records are fictional.

Repository and demonstration

GitHub repository: https://github.com/sanjana05240 /staffsync
Local application: http://localhost:5173
Demo recording: docs/demo/StaffSync-Demo.mp4

Requirements and functional design

Manual records and disconnected spreadsheets make it difficult to maintain a consistent view of employee information, leave decisions and attendance. StaffSync centralizes these workflows while preserving the authority and visibility appropriate to each organizational role.

Role	Primary responsibilities	Access boundary
HR	Create and edit employees; assign managers and roles; review leave; read attendance; publish updates and assign training.	Organization-wide access; own account cannot be deactivated or demoted through self-editing.
Manager	Read direct-team records and attendance; review team leave; publish announcements; assign training.	Direct reports only for team operations. Cannot approve own leave or review unrelated employees.
Employee	Maintain own profile; request leave; check in and out; read announcements; view assigned training and notifications.	Own personal records and explicitly assigned content.

Functional requirements

Authentication: employee registration, sign-in, session recovery and sign-out with server-side account checks.

People: searchable employee directory, manager assignment, roles, deactivation, personal details and CSV export.

Leave: valid date ranges, overlap detection, pending status, authorized review and status notification.

Attendance: one check-in per employee per India calendar day, a matching checkout and scoped history.

Communication and growth: announcement CRUD, manager training assignment and private in-app notifications.

Nonfunctional requirements

Input validation, bounded API responses, unique database indexes, responsive layouts, accessible form labels, visible error feedback and lazy-loaded feature screens support reliability and usability. Table pagination displays ten records per page. The application is a capstone implementation rather than a complete enterprise payroll or compliance system.

System architecture and user flows



JSON requests with JWT authentication; MongoDB persistence through Mongoose

Model view controller organization

The React client provides the view layer and uses fetch to call same-origin JSON endpoints. Express route files map requests to controllers. Authentication middleware validates JWTs, loads the current active user and checks role permissions. Controllers validate fields and apply ownership or team rules before calling Mongoose models. MongoDB stores the resulting documents.

Employee onboarding

HR creates an employee account, then assigns department, designation and manager. HR can promote an employee to manager when needed. Self-registration always creates an employee role; client-supplied privileged fields are ignored. An employee may change their own name and phone number but cannot grant themselves a new role.

Leave request lifecycle

An employee submits a date range and reason. The server rejects impossible dates, reversed ranges and overlap with an existing pending or approved request. A direct manager or HR may approve or reject a pending request. A conditional database update prevents a second review from overwriting an existing final decision. The employee receives a private notification.

Attendance and training flows

The server derives attendance identity from the authenticated account and derives the calendar day using India Standard Time. A compound unique index prevents duplicate daily check-ins. Checkout closes the employee's open record. Training creators select eligible people; a manager can only assign their own direct reports. Assigned employees receive notifications and see the session on their Training screen.

Interface structure

A consistent shell contains Overview, People, Leave requests, Attendance, Announcements and Training. The top bar exposes account details, notifications and sign-out. Each role sees scoped data and relevant actions. On mobile, navigation scrolls horizontally and wide tables remain inside their own scroll containers.

Database design

Collection	Important fields	Relationships
users	name, email, passwordHash, role, active	One authentication identity per employee; manager is a user role.
employees	user, department, designation, phone, manager	Unique user reference; optional manager reference to users.
leaves	employee, startDate, endDate, reason, status, reviewedBy, reviewedAt	Requester and reviewer reference users.
attendances	employee, date, checkedInAt, checkedOutAt	Unique combination of employee and date.
announcements	title, body, author	Author references the posting HR user or manager.
trainings	title, description, date, assignedTo, createdBy	Creator and assigned participants reference users.
notifications	recipient, title, body, section, readAt	Private recipient reference; linked application section.

Entity relationships

User 1 to 1 Employee. Manager User 1 to many Employee profiles. User 1 to many Leave requests and Attendance records. User 1 to many authored Announcements and created Training programs. Users and Training programs have a many-to-many assignment relationship through assignedTo. User 1 to many private Notifications.

Indexes and data retention

Email and employee-profile ownership have unique indexes. Attendance has a unique employee/date index. Manager references, leave employee references, training assignees and notification recipient/time fields are indexed. Employee removal deactivates the user instead of deleting historical HR records.

Persistence verification

The local demo starts the official MongoDB 7.0.24 executable through mongodb-memory-server as a process launcher. Despite the package name, the configured WiredTiger dbPath is on disk. A separate verification inserted a marker, stopped MongoDB, restarted it using the same directory and read the marker successfully. The demo is independent of the isolated disposable databases used by the test suite.

Implementation and security

API area	Representative routes
Authentication	POST /api/auth/register; POST /api/auth/login; GET /api/auth/me
Employees	GET/POST /api/employees; PATCH/DELETE /api/employees/:userId
Leave	GET/POST /api/leaves; PATCH /api/leaves/:id
Attendance	GET /api/attendance; POST /api/attendance/check-in; POST /api/attendance/check-out
Announcements	GET/POST /api/announcements; PATCH/DELETE /api/announcements/:id
Training	GET/POST /api/training
Notifications	GET /api/notifications; PATCH /api/notifications/:id; PATCH /api/notifications/read-all

Authentication and authorization

Passwords are hashed with a random salt and scrypt, and password hashes are excluded from JSON responses. JWTs use a fixed HS256 algorithm, issuer, audience and two-hour expiry. The authentication middleware reloads the user on every request, so account deactivation invalidates an already issued token. Login and registration are rate limited.

Input and application protections

Text fields have type and length checks. Calendar inputs require real ISO dates. Protected routes derive identity from the token, and controllers use explicit allowlists for editable fields. Helmet supplies standard security headers. JSON payload size is limited. Exported CSV cells are quoted and formula-leading characters are neutralized. Frontend text is rendered as React text rather than inserted as arbitrary HTML.

Implementation choices

The implementation uses native fetch, React state, hash navigation and scrypt in place of the reference stack's Axios, Redux, React Router and bcrypt examples. Managers share the users collection with a role field, avoiding a duplicated authentication identity. These choices retain the requested MERN architecture and functional boundaries. Notifications are delivered on a best-effort basis after the primary record is saved.

Dashboard and people management

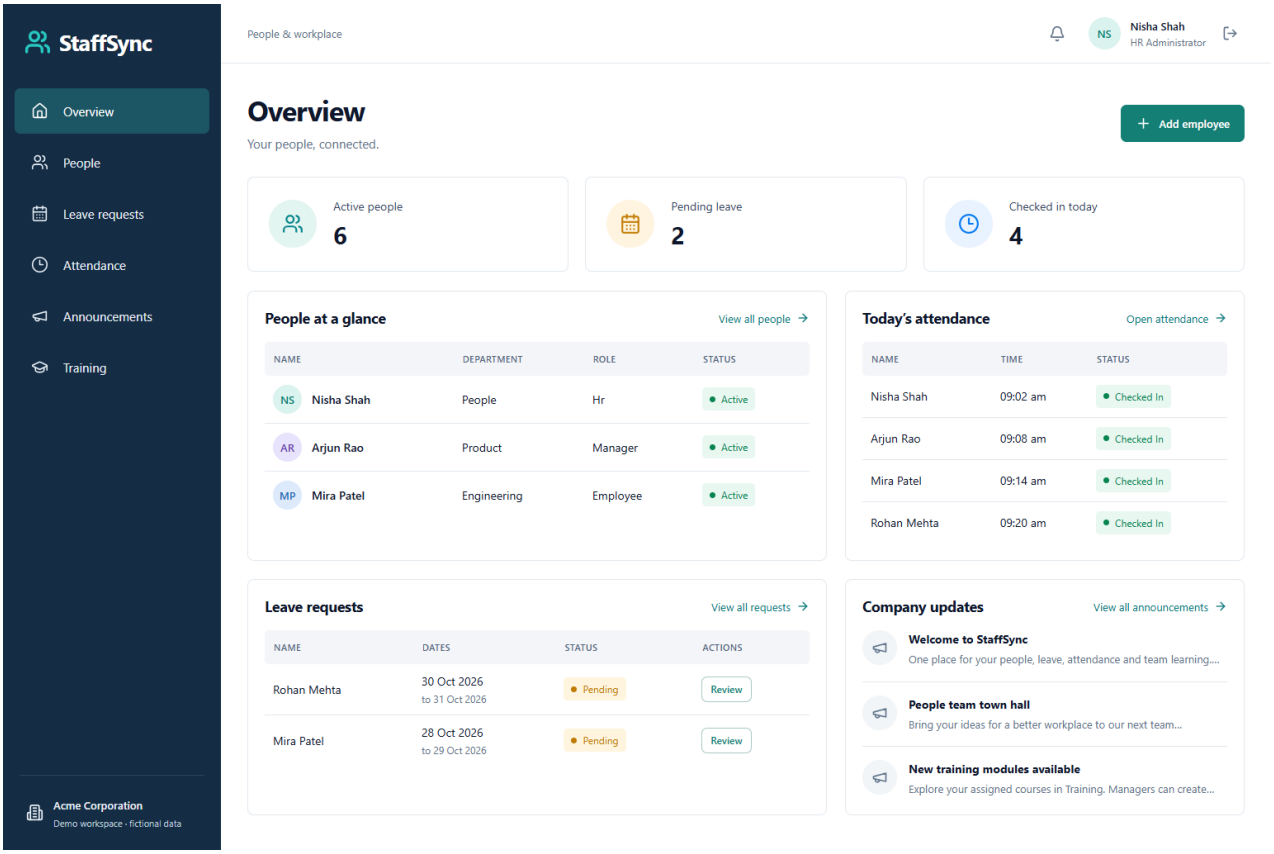


Figure 1. HR overview with live counts, people, attendance, leave and company updates.

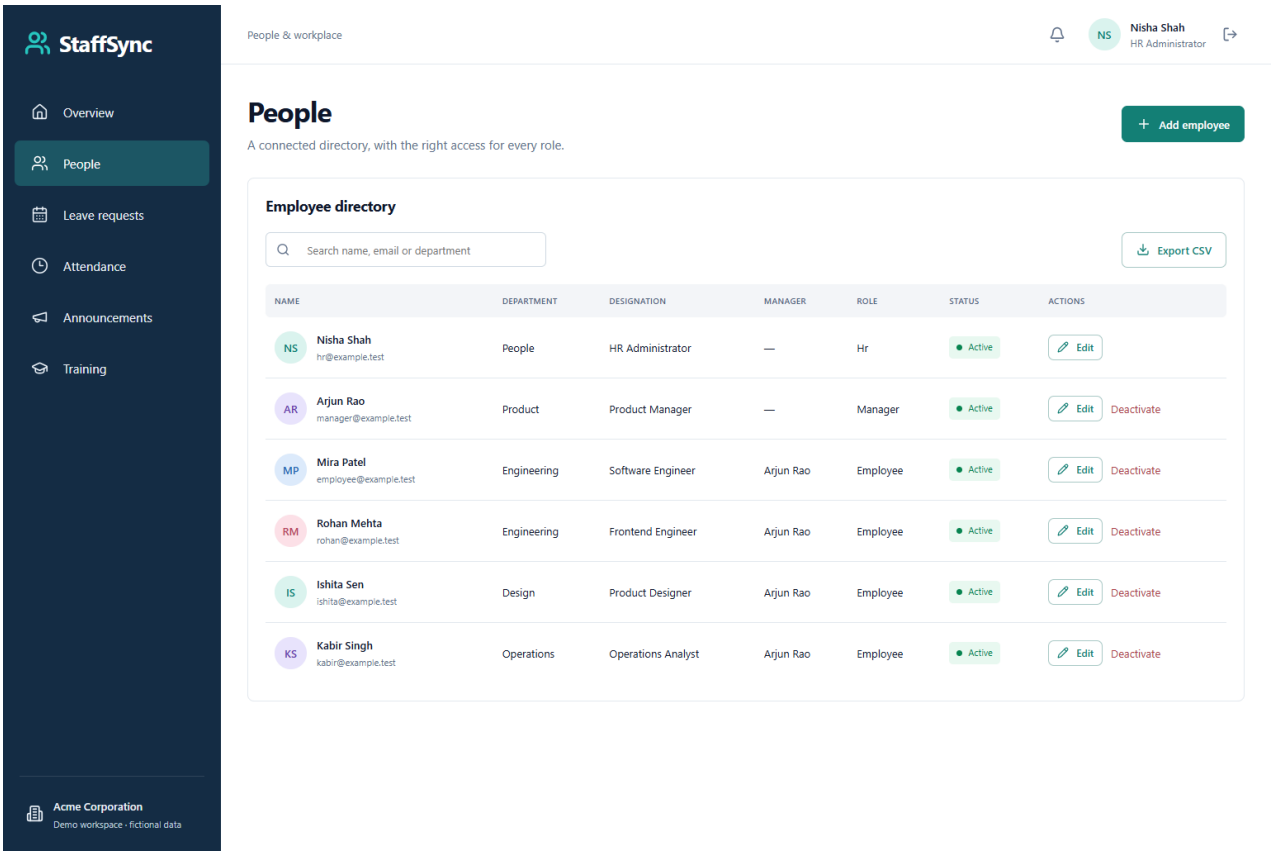


Figure 2. Searchable employee directory with manager assignment, editing, deactivation and export.

Leave and attendance management

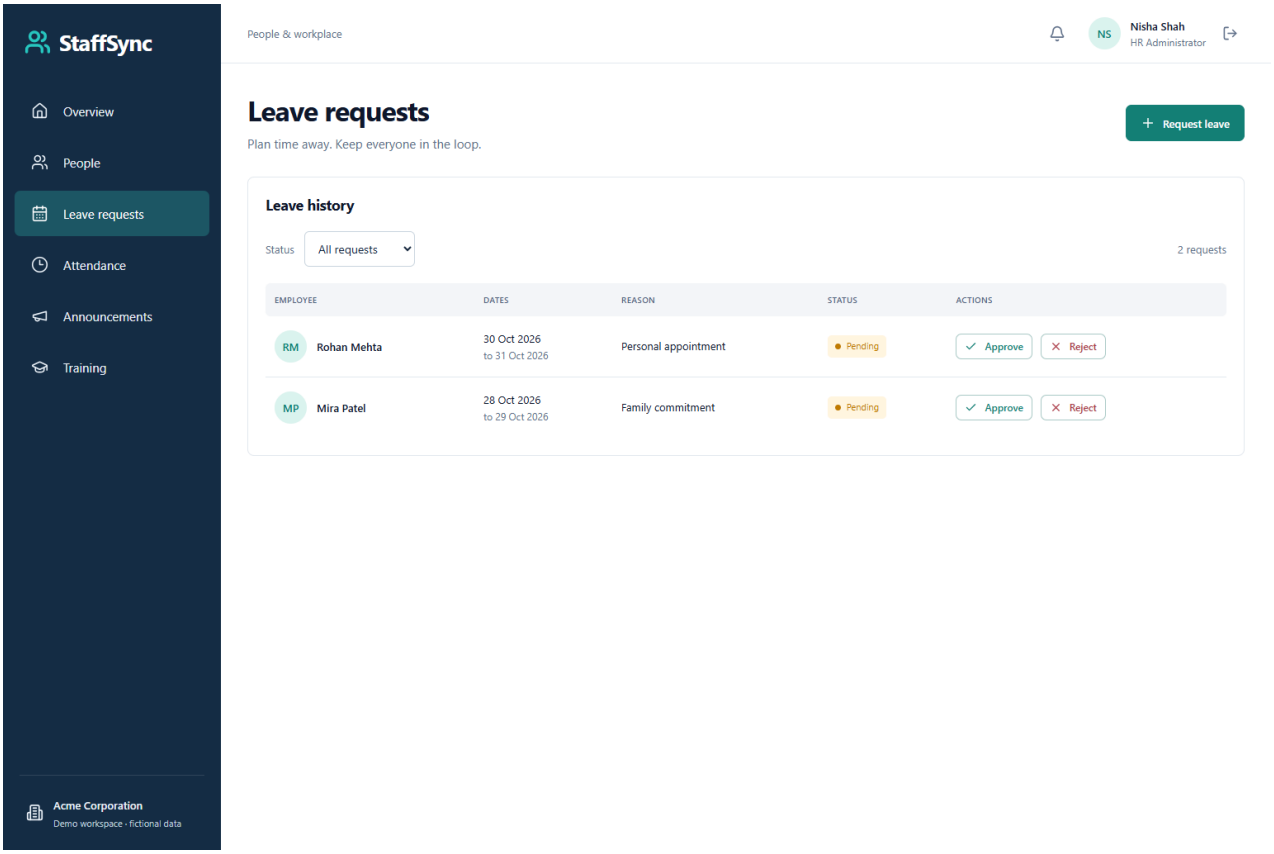


Figure 3. HR leave review. Employees see their own requests; managers see their direct team.

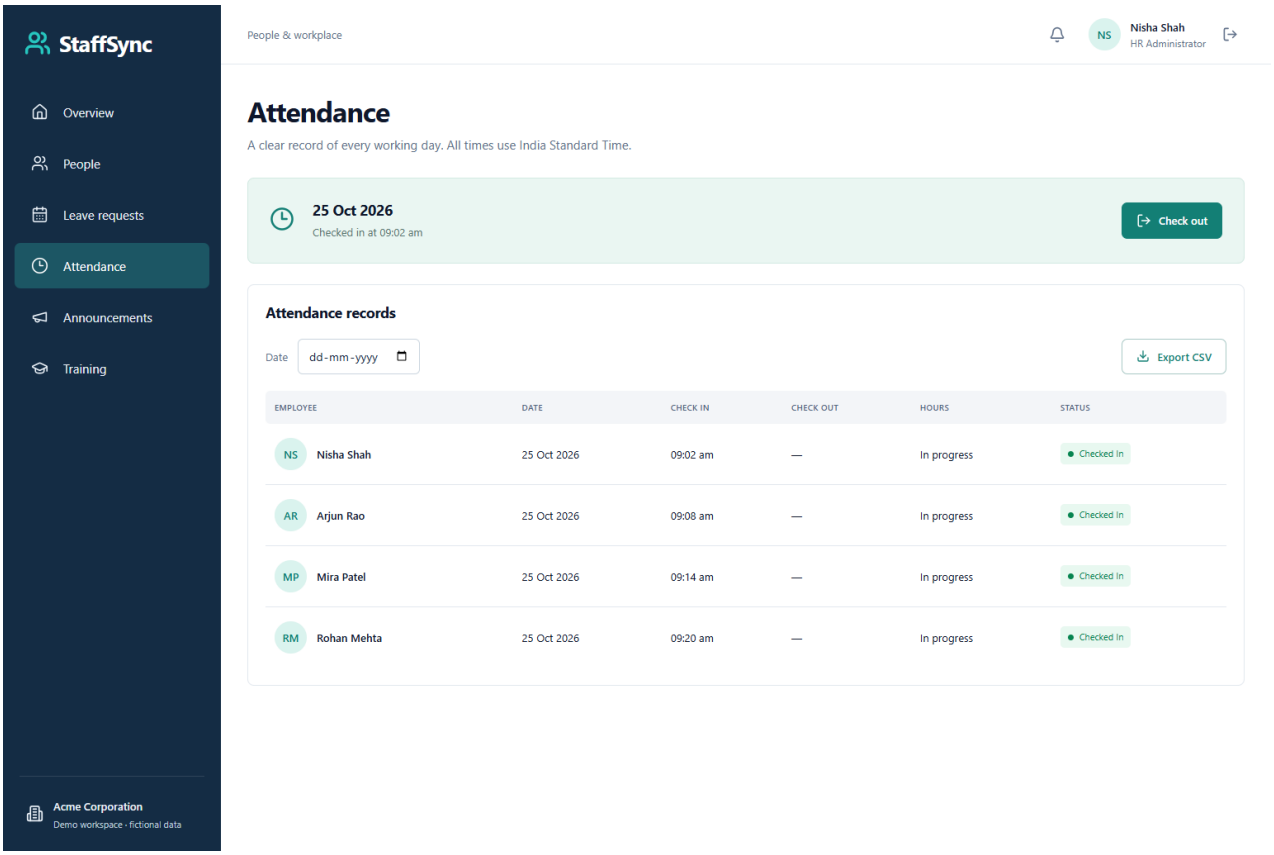


Figure 4. Attendance history and the authenticated account's daily check-in or checkout control.

Announcements and training

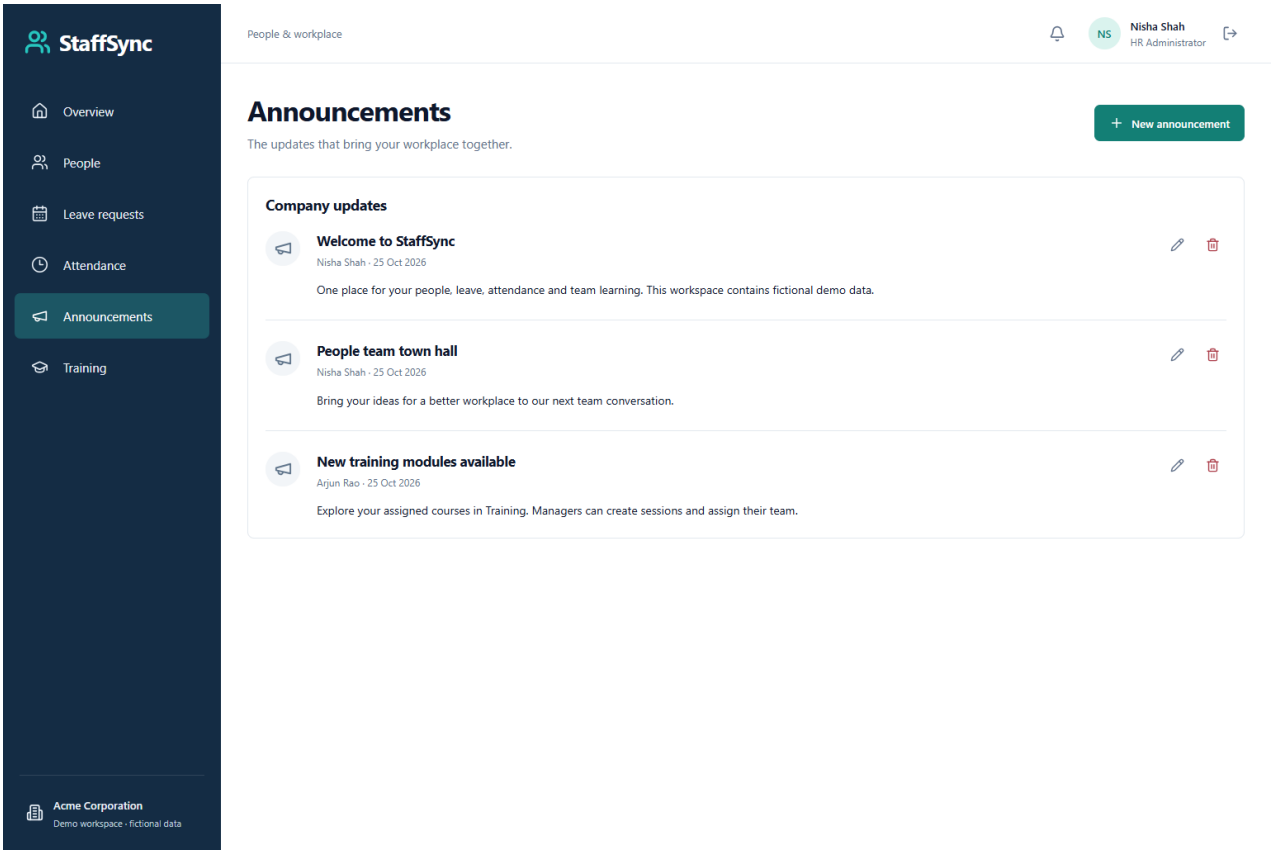


Figure 5. Company announcements with author-controlled editing and HR administration.

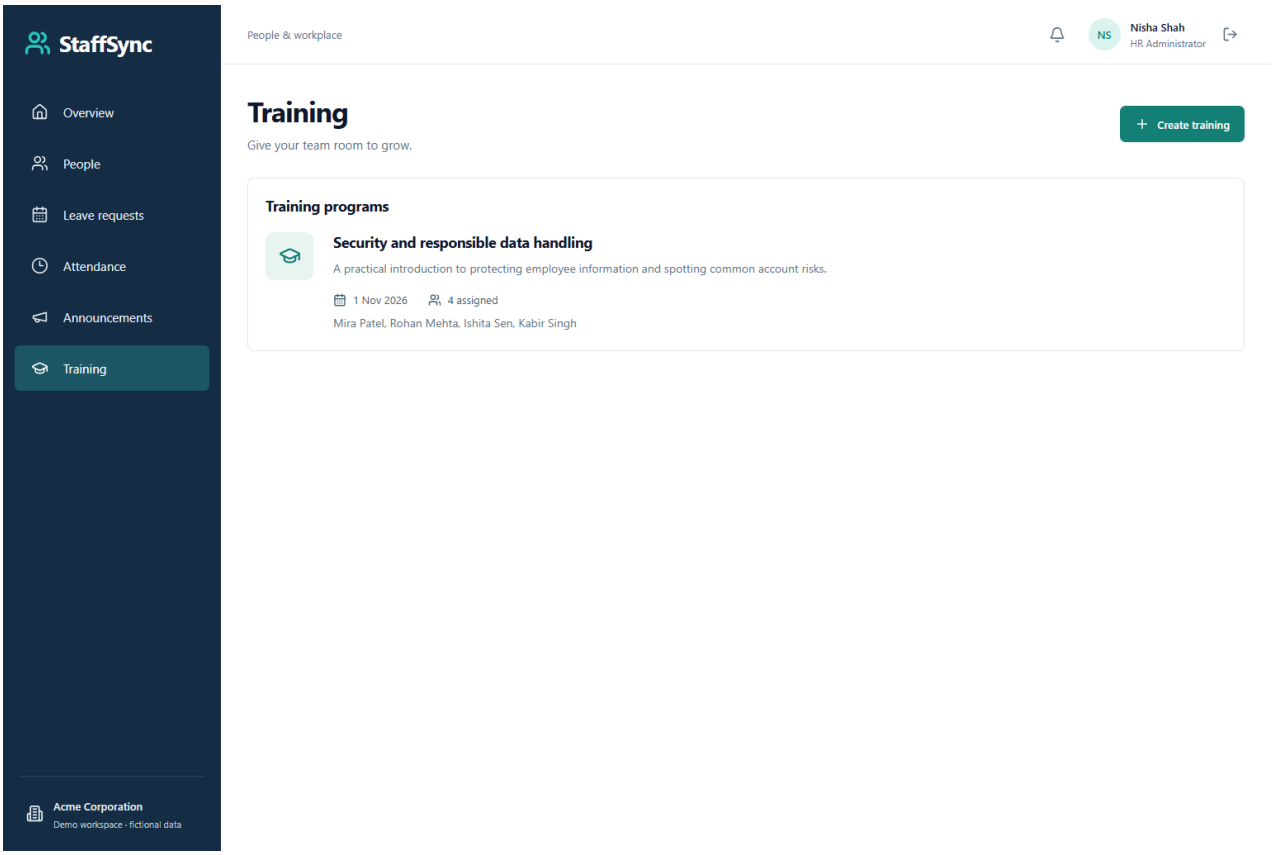


Figure 6. Training programs and team assignments backed by MongoDB documents.

Testing and observed results

The final backend suite completed with fourteen passing tests, no failures and no skipped tests. Browser mutation tests used a separate temporary MongoDB database so that verification did not alter the localhost demo records.

Verification	Observed result
Registration and authentication	Public registration cannot grant HR; duplicate email, anonymous access, invalid token and incorrect password are rejected.
Profile and team scope	Employee sees own profile; manager sees direct team; optional fields may be cleared; role escalation and cross-user edits are blocked.
Leave workflows	Invalid dates and overlap are rejected; manager approves direct-team leave; self review and a second final decision are blocked.
Attendance	India day boundaries, duplicate check-in, checkout conflicts and scoped records pass.
Announcements and training	Role/author checks pass; manager cannot assign an unrelated employee; assignees see their training.
Notifications	Only the recipient can read or mark a notification; mark-all-read works.
Browser workflows	HR creates and assigns a colleague; employee submits leave and checks in/out; manager reviews and assigns training; reload preserves session and records.
Exports and pagination	People CSV download succeeds; directory displays 10 records then 2 records across two pages.
Responsive and console checks	1536x1024 desktop, 1440x1000 interactions and 390x844 mobile; no document overflow or unexpected browser console errors.
MongoDB persistence	A marker survives a complete stop/start using the same WiredTiger data directory.

Testing boundaries

Verification covers the local capstone application and fictional accounts. It does not establish high-concurrency guarantees, large-organization performance, production security certification or mentor acceptance. API lists are bounded; frontend pagination operates within those returned records. Concurrent overlapping leave applications require a stronger transaction or locking design for strict exclusion.

Installation operation and conclusion

Local demonstration

Install Node.js 24, clone the repository and run `npm ci`. Run `npm run demo` to build the interface, start MongoDB and serve the app at http://localhost:5173. The first run downloads the official MongoDB executable; subsequent runs reuse the cache. Local records and the demo JWT secret remain under .data, which is excluded from Git.

Role	Fictional demo login
HR	hr@example.test
Manager	manager@example.test
Employee	employee@example.test

Demo password for these fictional accounts: `StaffSyncDemo2026!`. These credentials are only for the explicitly labeled demo and must not be used as production credentials.

Managed MongoDB deployment

Build the frontend with npm run build. Copy .env.example to .env and set MONGODB_URI, a strong random JWT_SECRET, and optional PORT. Create the initial HR account using the bootstrap command and remove bootstrap credentials afterward. The normal server connects to the configured MongoDB Community or Atlas instance. Before public hosting, configure TLS, network binding, secret management and database backups; replace public self-registration with organization-controlled onboarding if real employee data will be stored.

Conclusion and future scope

StaffSync implements the requested role-based HR workflows on the MERN stack, with verified MongoDB persistence, usable dashboards and actual browser-tested interactions. Future work includes password recovery, invitation workflows, transactional multi-document edits, server-side pagination, more detailed reporting, audit logs and stricter concurrent leave-booking rules.

References

SkillWallet Full Stack Development MERN Stack program instructions and StaffSync workspace, accessed 25 October 2026.

Supplied reference code folder: https://drive.google.com/drive/folders/1YfGcJk5b1lA5SBOEkmMDao2HwySMabc2

MongoDB documentation: https://www.mongodb.com/docs/ ; Mongoose: https://mongoosejs.com/docs/ ; Express: https://expressjs.com/ ; React: https://react.dev/ .

Repository documentation: README.md, docs/architecture.md, docs/validation.md and docs/source-materials.md.